

Technical Documentation

Pipeline Explanation

The pipeline was designed to simulate a real-time mobile application logging system using Apache NiFi, Apache Kafka, and Apache Hadoop.

The flow starts with continuously generated messy streaming data coming from a Python script. The generated logs intentionally contain missing values, duplicate records, invalid timestamps, corrupted values, and inconsistent numeric fields to simulate real-world streaming environments.

Inside NiFi, the data is ingested continuously, split into smaller chunks, transformed from CSV to JSON, cleaned, validated, deduplicated, published into Kafka topics, consumed again from Kafka, and finally stored into HDFS in a partitioned structure.

The main goal of the pipeline was to demonstrate stable streaming ingestion, real-time processing, data quality handling, Kafka integration, and scalable Hadoop storage.

Data Flow Explanation

- ListFile & FetchFile:

I used these processors to continuously monitor the incoming directory and ingest newly generated streaming files automatically.

- SplitText:

I used SplitText to divide incoming files into smaller chunks close to the required 64 KB limit. This improves memory handling and allows smoother streaming behavior inside NiFi.

- ConvertRecord:

This processor converts the CSV records into JSON format using CSVReader and JsonRecordSetWriter.

- QueryRecord (Cleaning Stage):

I used QueryRecord to clean invalid and incomplete data. This stage handled null values, invalid numeric fields, and malformed timestamps.

- QueryRecord (Timestamp Standardization):

Different timestamp formats were normalized into a unified format (yyyy-MM-dd HH:mm:ss). Invalid timestamps were replaced with the current processing timestamp.

- **DeduplicateRecord:**

This processor removes duplicated records before publishing them into Kafka. The duplicate check was based on log_id.

- **PublishKafka:**

After cleaning and validation, the final JSON records are published into the Kafka topic mobile_app_logs.

- **ConsumeKafka:**

A separate NiFi consumer continuously consumes the Kafka messages for downstream storage.

- **Content Type Handling:**

Some FlowFiles appeared in hexadecimal view because the mime.type attribute was missing. To fix this, I used an UpdateAttribute to do so and make them json files

- **PutHDFS:**

The final cleaned JSON records are stored inside HDFS using date-based partitioning.

Transformation Logic

The transformation stage focused on making the streaming data consistent and suitable for long-term storage and analytics.

- **Null Handling:**

I replaced missing values with default placeholders such as:

- UNKNOWN_USER
- UNKNOWN_DEVICE
- UNKNOWN
- 0.0.0

This was mainly done to keep the pipeline stable and avoid schema failures during streaming processing. In real-world production systems, this logic would usually be replaced with stricter validation rules or rejected records.

- **Timestamp Normalization:**

The generated data intentionally contained multiple timestamp formats such as:

2026-05-15 03:39:14

15/05/2026 03:39

INVALID_TIMESTAMP

I used QueryRecord SQL logic to standardize all timestamps into a single format:

yyyy-MM-dd HH:mm:ss

Invalid timestamps were replaced with the current processing timestamp.

- **Numeric Inconsistency Handling:**

Some records contained invalid or negative values inside session_duration. Invalid values were replaced with 0 to maintain schema consistency.

- **CSV to JSON Transformation:**

The pipeline converts all CSV records into structured JSON records before publishing to Kafka.

- **Deduplication:**

I used DeduplicateRecord to remove duplicate log_id values before the records enter Kafka. This helps reduce duplicate processing downstream.

Error Handling Approach

The pipeline includes multiple layers of error handling to improve reliability and stability.

- **Failure Relationships:**

Processors such as ConvertRecord, QueryRecord, and DeduplicateRecord route failed records into separate failure paths for monitoring and debugging.

- **Back Pressure:**

Back pressure was configured to prevent queue overflow when downstream systems become slower than the ingestion speed. This prevents excessive memory consumption inside NiFi.

- **Yield Duration:**

Processors were configured with Yield Duration to avoid aggressive retries during temporary failures. This helps reduce CPU overhead and stabilizes the flow.

- **Penalization:**

Failed FlowFiles are penalized temporarily before retrying. This gives external systems such as Kafka or HDFS time to recover from temporary issues.

- **At Least Once Delivery:**

Kafka consumption uses offset commits after successful processing. This guarantees that records are not lost, although duplicates may occasionally occur during failures or retries. DeduplicateRecord was added to reduce duplicate storage.

- **Monitoring:**

I used LogAttribute processors and queue monitoring to track FlowFiles, detect failures, and verify processor outputs during testing.

Partitioning Strategy

The final JSON data is stored in HDFS using date-based partitioning:

/data/year=2026/month=05/day=15/

This partitioning strategy improves:

- Query performance
- Data organization
- Scalability
- Easier downstream analytics

The partition values are generated dynamically inside NiFi using Expression Language functions.

Challenges Faced During Implementation

- Handling multiple timestamp formats inside streaming records was one of the main challenges.
- QueryRecord and UpdateRecord occasionally produced schema-related errors because the generated data intentionally contained inconsistent types and malformed values.
- Deduplication behavior in NiFi required understanding the difference between FlowFile-level duplication and Record-level duplication.
- Kafka delivery semantics required careful configuration to avoid data loss while maintaining stable processing behavior.
- Converting large numbers of small records into HDFS efficiently required batching and chunking strategies to avoid the small files problem.

- Managing malformed and corrupted rows while keeping the pipeline continuously running required separate failure handling paths and monitoring queues carefully.